

 Państwowa Akademia Nauk Stosowanych w Krńńia		Kierunek Informatyka	
Instrukcja do ćwiczń laboratoryjnych nr:	4	Nazwa przedmiotu: Programowanie w języku Kotlin	
Temat: Jetpack Ccompose – Kotlin – Android Studio		Tryb studiów: stacjonarne	
		Czas trwanie ćw. 2x45 min	
Autor materiałów: dr Marcin Skuba			

1. Treści programowe:

Programowanie deklaratywne, tworzenie interfejsu UI – Jetpack Compose, Kotlin

2. Cel zajęć:

Celem zajęć jest zrozumienie zasad oraz poznanie składni programowania deklaratywnego na przykładzie biblioteki JetPpack Compose do tworzenia interfejsów użytkownika w języku Kotlin.

3. Materiały dydaktyczne

Jetpack Compose to największa rewolucja w tworzeniu UI na Androida od czasu powstania tego systemu. Przejście z XML na Compose to nie tylko zmiana języka, to całkowita zmiana filozofii myślenia o kodzie.

Oto najważniejsze punkty, które musisz znać:

1. Programowanie deklaratywne vs imperatywne

To klucz do zrozumienia Compose.

- W starym stylu (imperatywnym) pisałeś instrukcje krok po kroku: "Znajdź przycisk, zmień mu kolor na czerwony, ustaw tekst na 'Kliknięto'". Musiałeś sam zarządzać stanem i dbać, by UI nadążało za logiką.
- W Compose (deklaratywnym) po prostu opisujesz, jak UI ma wyglądać w danym stanie: "Jeśli przycisk jest kliknięty, ma być czerwony i mieć napis 'Kliknięto'". Ty dostarczasz dane, a Compose zajmuje się resztą.

2. UI jako funkcja stanu

W Compose interfejs użytkownika jest wynikiem funkcji. Można to zapisać wzorem:

$$UI = f(State)$$

Gdy zmienia się State (stan), Compose automatycznie wywołuje funkcję ponownie z nowymi danymi. Ten proces nazywamy **Rekmpozycją** (Recomposition).

3. Kompozycja zamiast dziedziczenia

W starym systemie każdy widok (**View**) był gigantycznym obiektem dziedziczącym po tysiącach linii kodu. W **Compose** budujesz UI z małych, niezależnych klocków – funkcji **@Composable**.

Możesz stworzyć własny przycisk MyGreenButton i używać go wielokrotnie, łącząc go z innymi komponentami jak klocki LEGO.

4. Koniec z plikami XML

Cały interfejs pisziesz w Kotlinie. Oznacza to:

- Pełne wsparcie programistyczne (podpowiadanie składni, sprawdzanie typów).
- Możliwość używania pętli **if** czy **for** bezpośrednio wewnątrz definicji UI (np. "jeśli lista jest pusta, wyświetl komunikat, w przeciwnym razie wyświetl listę").

- Brak konieczności przełączania się między plikami .kt a .xml.

5. Recomposition (Inteligentne odświeżanie)

Compose jest bardzo sprytny. Kiedy zmienia się jedna mała dana (np. liczba polubień pod postem), nie rysuje całego ekranu od nowa. Odświeża tylko te konkretne funkcje @Composable, które używają tej zmienionej danej. Dzięki temu aplikacje są płynne i wydajne.

W programowaniu deklaratywnym nie mówisz programowi "jak" ma zmienić widok, ale "co" ma być wyświetlone w oparciu o aktualne dane.

Klasa aktywności:

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.example.mpje_jpc3.ui.theme.Mpje_JPC3Theme

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Mpje_JPC3Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true, showSystemUi = true)
@Composable
fun GreetingPreview() {
    Mpje_JPC3Theme {
        Greeting("Android")
    }
}
```

```
}
```

```
}
```

1. enableEdgeToEdge()

Ta funkcja sprawia, że Twoja aplikacja zajmuje **cały ekran**, łącznie z obszarami pod paskiem stanu (tam, gdzie masz godzinę i baterię) oraz paskiem nawigacji na dole.

- Dzięki temu interfejs wygląda nowocześnie i profesjonalnie.
- Gdy to włączysz, musisz używać tzw. *Window Insets* (np. `Modifier.safeDrawingPadding()`), żeby tekst nie "wszedł" pod zegarek lub kamerę.

2. setContent { ... }

To jest most łączący "stary" świat Activity z "nowym" światem Compose.

- W tradycyjnym Androidzie używało się `setContentView(R.layout.main)`.
- W Compose używamy `setContent`, aby powiedzieć systemowi: "od tego miejsca UI będzie budowane za pomocą funkcji `@Composable`, a nie plików XML".

3. Moje_JPC3Theme { ... }

To jest tzw. **Wrapper tematu** (nazwa pochodzi od Twojego projektu, np. *Moje JetPack Compose 3*).

- To tutaj definiujesz kolory (Primary, Secondary), czcionki i kształty dla całej aplikacji.
- Wszystko, co wrzucisz do środka tych klamer, automatycznie przejmie styl zdefiniowany w pliku `Theme.kt`. Dzięki temu zmiana koloru w jednym miejscu zmieni wygląd całej aplikacji.

4. Scaffold

Po angielsku to "rusztowanie". Jest to niesamowicie pomocny komponent, który daje Ci gotową strukturę ekranu zgodną z Material Design.

- Posiada specjalne "sloty" na: **TopBar** (pasek u góry), **BottomBar** (menu na dole) oraz **FloatingActionButton** (okrągły przycisk akcji).
- Scaffold dba o to, by te elementy nie nachodziły na siebie i miały odpowiednie odstępy.

5. @Composable

To najważniejsza adnotacja w Compose. Mówi ona kompilatorowi: "Ta funkcja nie jest zwykłą funkcją – ona służy do generowania interfejsu użytkownika".

- Funkcje te muszą zaczynać się z **wielkiej litery** (np. `fun Greeting`).
- Można je wywoływać tylko z poziomu innych funkcji `@Composable`.

6. @Preview(showBackground = true)

To narzędzie dla programisty, które pozwala zobaczyć wygląd komponentu **bez uruchamiania aplikacji** na telefonie czy emulatorze.

- `showBackground = true` sprawia, że w podglądzie zobaczysz białe tło (domyślnie jest przezroczyste, co utrudnia pracę).
- `showSystemUi = true` sprawia że widzimy cały ekran np. telefonu.
- Możesz mieć wiele podglądów, np. jeden dla trybu jasnego, a drugi dla ciemnego.

7. fun GreetingPreview() { ... }

To po prostu zwykła funkcja, która służy wyłącznie do wyświetlenia podglądu.

- Zazwyczaj wewnątrz niej wywołujesz swój temat (Moje_JPC3Theme) i konkretną funkcję UI, którą właśnie tworzysz (np. Greeting("Android")).
- Ta funkcja nie trafia do finalnej aplikacji na telefonie – służy tylko Tobie w Android Studio.

8. Nawiasy okrągłe () – Parametry (Właściwości)

Wewnątrz () przekazujemy **argumenty funkcji**. To tutaj decydujesz, jak dany komponent ma wyglądać lub jak ma się zachowywać "technicznie".

Np.: Tekst do wyświetlenia, kolory, kształty, style czcionek oraz najważniejszy element w Compose: **Modifier** (modyfikator).

- **Przykłady:**
 - W Text(...) – treść napisu, kolor, wielkość liter.
 - W Button(...) – reakcja na kliknięcie (onClick).

9. Nawiasy klamrowe {} – Treść (Sloty / Trailing Lambda)

Wewnątrz {} umieszczamy to, co ma się znaleźć **w środku** danego komponentu. W Kotlinie, jeśli ostatnim parametrem funkcji jest inna funkcja (tzw. lambda), możemy ją wyciągnąć poza nawiasy okrągłe.

Wpisujemy tam inne komponenty @Composable.

- **Przykłady:**
 - W Button { ... } – tutaj zazwyczaj wrzucasz Text, aby przycisk miał jakąś etykietę, albo Icon.
 - W Column { ... } – tutaj wrzucasz listę elementów, które mają być jeden pod drugim.

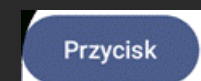
Podstawowe elementy do budowy interfejsu:

• Proste pole tekstowe:

```
Text(
    text = "Tekst przycisku",
    color = Color(0,0,255)
)
```



• Prosty przycisk z etykietą tekstową



```
Button(onClick = {
    Log.d("Click", "Kliknięcie")
}) {
    Text(text = "Przycisk")
}
```

• Modifier (modyfikator)

Modifier to absolutnie najważniejsze narzędzie w Jetpack Compose. Jeśli Composable (np. Text lub Button) jest "rzeczą", to Modifier jest opisem tego, jak ta rzecz ma wyglądać, gdzie ma stać i jak ma reagować na dotyk.

Możesz o nim myśleć jak o liście instrukcji, którą dołączasz do elementu.

Modyfikatory pozwalają na:

- Zmianę wyglądu: tło (**background**), obramowanie (**border**), kształt (**clip**).
- Ustawienie rozmiaru: szerokość i wysokość (**width**, **height**, **fillMaxSize**).
- Dodanie odstępów: marginesy wewnętrzne (**padding**).
- Interakcję: reagowanie na kliknięcia (**clickable**), przesuwanie (**scrollable**).
- Pozycjonowanie: przesunięcie względem innych elementów (**offset**).

Kluczowa zasada: Kolejność ma znaczenie!

To najczęstszy błąd początkujących. Modifiers są przetwarzane od góry do dołu (jeden po drugim).

Zmiana kolejności całkowicie zmienia wynik.

Przykład:

1. **.padding(16.dp).background(Color.Red)** – najpierw robisz margines, a potem malujesz tło (tło będzie tylko pod tekstem).
2. **.background(Color.Red).padding(16.dp)** – najpierw malujesz tło, a potem dodajesz margines wewnątrz tego tła (cały czerwony prostokąt będzie większy).

```
Text(  
    text = "Cześć!",  
    modifier = Modifier  
        .fillMaxWidth()           // 1. Rozciągnij na całą szerokość  
        .background(Color.Yellow) // 2. Daj żółte tło  
        .padding(20.dp)           // 3. Dodaj odstęp wewnątrz  
        .clickable { Log.d("Click", "Kliknięcie") } // 4. Spraw, by tekst był klikalny  
)
```



Wybrane Atrybuty Modifier:

1. Rozmiar i Wymiary (Size)

Te modyfikatory decydują o tym, ile miejsca na ekranie zajmie Twój komponent.

Atrybut	Opis	Przykład
fillMaxSize()	Zajmuje całą dostępną przestrzeń (szerokość i wysokość).	Modifier.fillMaxSize()
fillMaxWidth()	Rozciąga element na całą dostępną szerokość.	Modifier.fillMaxWidth()
fillMaxHeight()	Rozciąga element na całą dostępną wysokość.	Modifier.fillMaxHeight()
size(dp)	Ustawia stałą szerokość i wysokość (np. 100.dp na 100.dp).	Modifier.size(100.dp)
width(dp) / height(dp)	Ustawia konkretną szerokość lub wysokość.	Modifier.width(50.dp)
wrapContentSize()	Pozwala elementowi mieć własny rozmiar, ignorując limity rodzica.	Modifier.wrapContentSize()

2. Odstęp i Pozycjonowanie (Layout)

Decydują o "oddechu" wokół elementu i jego miejscu w kontenerze.

Atrybut	Opis	Przykład
<code>padding(dp)</code>	Dodaje margines wewnętrzny wokół elementu.	<code>Modifier.padding(16.dp)</code>
<code>offset(x, y)</code>	Przesuwa element o daną wartość bez zmiany układu innych.	<code>Modifier.offset(x = 10.dp)</code>
<code>weight(float)</code>	(Tylko w Row/Column) Rozdziela wolną przestrzeń proporcjonalnie.	<code>Modifier.weight(1f)</code>
<code>align(alignment)</code>	Ustawia wyrównanie elementu wewnątrz Box, Row lub Column.	<code>Modifier.align(Alignment.Center)</code>

3. Wygląd i Stylizacja (Graphics)

Wszystko, co sprawia, że komponent wygląda ładnie.

Atrybut	Opis	Przykład
<code>background(color, shape)</code>	Ustawia kolor tła lub gradient oraz opcjonalnie kształt.	<code>Modifier.background(Color.Red)</code>
<code>border(width, color, shape)</code>	Dodaje obramowanie wokół elementu.	<code>Modifier.border(2.dp, Color.Black)</code>
<code>clip(shape)</code>	Przycina element do danego kształtu (np. koła).	<code>Modifier.clip(CircleShape)</code>
<code>alpha(float)</code>	Ustawia przezroczystość (od 0.0 do 1.0).	<code>Modifier.alpha(0.5f)</code>
<code>shadow(elevation, shape)</code>	Dodaje cień pod elementem (efekt uniesienia).	<code>Modifier.shadow(4.dp)</code>

4. Interakcje i Zdarzenia (Interactions)

Sprawiają, że UI reaguje na działania użytkownika.

Atrybut	Opis	Przykład
<code>clickable { }</code>	Reaguje na kliknięcie i dodaje efekt "fali" (ripple).	<code>Modifier.clickable { doSomething() }</code>
<code>combinedClickable</code>	Obsługuje kliknięcie, podwójne kliknięcie i długie przytrzymanie.	<code>Modifier.combinedClickable { ... }</code>
<code>scrollable(...)</code>	Pozwala na przewijanie elementu gestem.	<code>Modifier.scrollable(...)</code>
<code>selectable(...)</code>	Pozwala na zaznaczenie (np. w RadioButton).	<code>Modifier.selectable(...)</code>

5. Bezpieczne obszary (Insets)

Modyfikatory, o które pytałeś wcześniej, dbające o widoczność UI.

Atrybut	Opis	Przykład
<code>safeDrawingPadding()</code>	Omija paski systemowe, wycięcia i klawiaturę.	<code>Modifier.safeDrawingPadding()</code>
<code>statusBarsPadding()</code>	Odsuwa tylko od górnego paska stanu.	<code>Modifier.statusBarsPadding()</code>
<code>navigationBarsPadding()</code>	Odsuwa tylko od dolnego paska nawigacji.	<code>Modifier.navigationBarsPadding()</code>

Obsługa zdarzeń Modifier.combinedClickable { ... }

Modifier.combinedClickable to "starszy brat" zwykłego .clickable. Pozwala on Twojemu komponentowi reagować na trzy różne rodzaje interakcji: **zwykłe kliknięcie**, **podwójne kliknięcie** oraz **długie przytrzymanie**.

Jest to idealne rozwiązanie do elementów listy, gdzie np. kliknięcie otwiera szczegóły, a długie przytrzymanie otwiera menu usuwania.

Modyfikator ten jest oznaczony jako **Experimental**. Aby go użyć, musisz dodać nad swoją funkcją adnotację `@OptIn(ExperimentalFoundationApi::class)`.

Oto jak stworzyć element, który reaguje na różne gesty:

```
@OptIn(ExperimentalFoundationApi::class) // Wymagane dla combinedClickable
@Composable
fun GestureBox() {
    val context = LocalContext.current

    Box(
        modifier = Modifier
            .size(200.dp)
            .background(Color.LightGray, RoundedCornerShape(16.dp))
            .combinedClickable(
                onClick = {
                    Toast.makeText(context, "Kliknięto!", Toast.LENGTH_SHORT).show()
                },
                onDoubleClick = {
                    Toast.makeText(context, "Podwójne kliknięcie!",
Toast.LENGTH_SHORT).show()
                },
                onLongClick = {
                    Toast.makeText(context, "Długie przytrzymanie!",
Toast.LENGTH_SHORT).show()
                }
            ),
        contentAlignment = Alignment.Center
    ) {
        Text("Testuj Gesty")
    }
}
```



- **Column**

Column to jeden z trzech podstawowych układów (layoutów) w Jetpack Compose. Jej zadaniem jest układanie elementów **pionowo** – jeden pod drugim.

Wszystko, co wrzucisz do środka klamer {} w Column, pojawi się na ekranie w kolejności od góry do dołu. Jest to odpowiednik pionowego **LinearLayout** ze starego systemu XML.

Pozycjonowanie (Alignment i Arrangement)

Column pozwala Ci kontrolować, jak elementy mają się zachowywać wewnątrz niej:

- **VerticalArrangement** (Odstępy pionowe): Decydujesz, czy elementy mają być zbite na górze, na środku, na dole, czy może rozłożone równomiernie (np. **Arrangement.SpaceBetween**).
- **HorizontalAlignment** (Wyrównanie poziome): Decydujesz, czy elementy mają być przyklejone do lewej krawędzi, wyśrodkowane poziomo, czy do prawej (np. **Alignment.CenterHorizontally**).

```
Column(
    modifier = Modifier.fillMaxSize(),
    verticalArrangement = Arrangement.Center,           // Wyśrodkuj w pionie
    horizontalAlignment = Alignment.CenterHorizontally // Wyśrodkuj w poziomie
) {
    Text("Element 1")
    Text("Element 2")
    Button(onClick = { /* ... */ }) {
        Text("Przycisk pod tekstami")
    }
}
```



Domyślnie **Column** nie jest przewijalna. Jeśli dodasz do niej 50 przycisków, te, które nie zmieszczą się na ekranie, po prostu zostaną ucięte. Aby lista była przewijalna, musisz dodać modyfikator:

Modifier.verticalScroll(rememberScrollState()) lub użyć komponentu **LazyColumn** (odpowiednik **RecyclerView**).

- **Row**

Row to poziomy odpowiednik **Column**. Jest to kontener, który układa wszystkie elementy wewnątrz klamer {} jeden obok drugiego (w poziomie).

Możesz o nim myśleć jak o rzędzie w tabeli lub poziomym **LinearLayout** ze starego systemu XML.

Wszystko, co dodasz do **Row**, pojawi się od lewej do prawej strony ekranu. Jest to idealny układ, gdy chcesz umieścić np. ikonę obok tekstu lub dwa przyciski obok siebie.

Główne parametry (wewnątrz nawiasów okrągłych)

Podobnie jak w **Column**, tutaj też mamy dwa kluczowe ustawienia, które decydują o tym, jak elementy "pływają" wewnątrz rzędu:

- **horizontalArrangement** (Rozmieszczenie w poziomie):
 - **Arrangement.Start** (domyślne): wszystko na lewo.

- **Arrangement.End**: wszystko na prawo.
- **Arrangement.Center**: na środku.
- **Arrangement.SpaceBetween**: pierwszy element na lewo, ostatni na prawo, reszta rozłożona równo.
- **verticalAlignment** (Wyrównanie w pionie):
 - Ponieważ rząd ma określoną wysokość, możesz zdecydować, czy elementy mają być przyklejone do góry (**Alignment.Top**), wyśrodkowane pionowo (**Alignment.CenterVertically**), czy na dole (**Alignment.Bottom**).

```
Row(
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp)
        .background(Color.LightGray),
    verticalAlignment = Alignment.CenterVertically, // Wyśrodkuj ikonę i tekst w pionie
    horizontalArrangement = Arrangement.spacedBy(8.dp) // Dodaj 8dp odstępu między
    elementami
) {
    Icon(Icons.Default.Info, contentDescription = null)
    Text(text = "Masz nową wiadomość!")
}
```

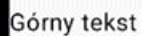
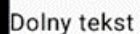


• Spacer

Spacer to najprostszy, a jednocześnie jeden z najczęściej używanych komponentów w Jetpack Compose. Jak sama nazwa wskazuje, służy do tworzenia **pustej przestrzeni** (odstępu) między innymi elementami.

Możesz o nim myśleć jak o „niewidzialnym klocku”, który wypycha inne komponenty.

```
Column {
    Text("Górny tekst")
    Spacer(modifier = Modifier.height(20.dp)) // 20dp odstępu w pionie
    Text("Dolny tekst")
}
```

• Button

```
Button(
    onClick = { Log.d("Log", "Kliknięcie") },
    colors = ButtonDefaults.buttonColors(
        containerColor = Color(0xFFFFF5722), // Kolor tła przycisku
        contentColor = Color.White           // Kolor tekstu i ikon wewnątrz
    )
) {
    Text(text = "Przycisk")
    Spacer(modifier = Modifier.width(5.dp))
    Icon(Icons.Default.Info, contentDescription = null)
}
```

Wstrzykiwanie parametrów do funkcji:

Zamiast pisać ten sam kod przycisku wiele razy, tworzysz jedną funkcję i "wstrzykujesz" do niej to, co ma się zmieniać.

Oto przykład profesjonalnie napisanej funkcji `@Composable`, która przyjmuje tekst, ikonę, akcję kliknięcia oraz modyfikator.

```
@Composable
fun CustomIconButton(
    text: String,                // Wstrzykujemy tekst
    icon: ImageVector,          // Wstrzykujemy ikonę
    onClick: () -> Unit,        // Wstrzykujemy funkcję (akcję)
    modifier: Modifier = Modifier // Wstrzykujemy modyfikator (z domyślną wartością)
) {
    Button(
        onClick = onClick,
        modifier = modifier // Przekazujemy otrzymany modyfikator dalej
    ) {
        Icon(
            imageVector = icon,
            contentDescription = null // null, bo tekst obok już opisuje przycisk
        )
        Spacer(modifier = Modifier.width(8.dp)) // Odstęp między ikoną a tekstem
        Text(text = text)
    }
}
```

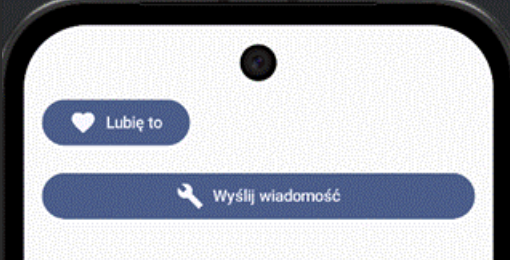
Dzięki temu, że "wstrzyknęliśmy" te parametry, możesz użyć tego samego przycisku w zupełnie różnych konfiguracjach:

```
@Composable
fun MyScreen() {
    Spacer(modifier = Modifier.height(45.dp))
    Column(modifier = Modifier.padding(16.dp)) {

        // 1. Standardowy przycisk "Lubię to"
        CustomIconButton(
            text = "Lubię to",
            icon = Icons.Default.Favorite,
            onClick = { Log.d("Click", "Kliknięto 'Lubię to'") }
        )

        Spacer(modifier = Modifier.height(16.dp))

        // 2. Ten sam przycisk, ale z dodatkowym modyfikatorem (cała szerokość)
        CustomIconButton(
            text = "Wyślij wiadomość",
            icon = Icons.Default.Build,
            onClick = { Log.d("Click", "Wysłano wiadomość") },
            modifier = Modifier.fillMaxWidth() // Nadpisujemy domyślny modyfikator
        )
    }
}
```



Zapamiętywanie stanu:

Przykład kodu, który inkrementuje wartość zmiennej oraz próbuje ją wyświetlić w polu tekstowym oraz wyświetla ją jako Log.

```
var count= 0
@Composable
fun MyButton(){
    Button(onClick = {
        count++
        Log.d("Click", "$count") //Działa
    }) {
        Text(text = "Count $count") //Nie działa
    }
}
```

Dlaczego ten przykład nie działa?

Twój kod nie działa (licznik w Log.d rośnie, ale na ekranie stoi w miejscu), ponieważ funkcja **MyButton nie wie, że musi się narysować od nowa**

✓ **Brak "Recompozycji" (Odświeżania)**

- W Compose interfejs użytkownika jest odświeżany tylko wtedy, gdy zmieni się coś, co **Compose obserwuje**. Twoja zmienna **var count** to zwykła zmienna procesora. Kiedy ją zmieniasz, system operacyjny o tym wie, ale **silnik Compose nie ma pojęcia, że ta zmiana powinna wpłynąć na wygląd ekranu**.
- Aby **Compose** wiedział, że ma odświeżyć funkcję (czyli wykonać tzw. **Recompozycję**), musisz użyć specjalnego typu opakowania: **MutableState**.
- Gdybyś nawet użył stanu, ale nie użył funkcji **remember**, to przy każdej próbie odświeżenia ekranu Twoja zmienna **count** byłaby tworzona od zera i ustawiana na 0. Funkcje w **Compose** są wykonywane wielokrotnie – muszą więc mieć "pamięć".

```
@Composable
fun MyButton(){
    var count by remember {mutableStateOf(0)}
    Button(onClick = {
        count++
        Log.d("Click", "$count") //Działa
    }) {
        Text(text = "Count $count") //Nie działa
    }
}
```

Dlaczego to teraz działa?

1. **mutableStateOf(0)**: Tworzy specjalny obiekt "obserwowalny". Gdy jego wartość się zmienia, **Compose** automatycznie odnajduje wszystkie miejsca, gdzie ta zmienna jest używana (u Ciebie w **Text**) i tylko te miejsca rysuje od nowa.

2. **remember { ... }**: Mówi Compose: "Przy pierwszym uruchomieniu stwórz tę zmienną, ale przy każdym kolejnym (rekompozycji) nie twórz jej na nowo, tylko weź tę, którą już znasz". Bez tego licznik zawsze wracałby do zera.
3. **by (Delegat)**: To ułatwienie w Kotlinie. Dzięki niemu możesz używać **count** jak zwykłej liczby (**count++**), zamiast pisać **count.value++**.

W powyższym przykładzie **var count** było poza funkcją **@Composable**. To bardzo zła praktyka w **Compose**, ponieważ:

- Jeśli obrócisz telefon, cała aktywność zostanie zniszczona i stworzona od nowa – Twoja zmienna może zostać zresetowana, chyba że użyjesz **ViewModel** lub **rememberSaveable** zamiast **remember**

```
var count by rememberSaveable {mutableStateOf(0)}
```

- Jeśli użyjesz tego samego przycisku w dwóch miejscach na ekranie, oba będą zmieniać tę samą, jedną zmienną, co prowadzi do błędów w logice.

UI w Compose to **lustrzane odbicie stanu**. Jeśli chcesz, żeby UI się zmieniło, musisz zmienić **State**, a nie zwykłą zmienną.

Kolejne elementy do budowy interfejsu:

- **Box**

Box to trzeci z fundamentów układów w Jetpack Compose (obok Column i Row). Jeśli Column to pion, a Row to poziom, to Box służy do układania elementów jeden na drugim (w głąb ekranu). Działa bardzo podobnie do **FrameLayout** ze starego systemu XML.

1. Nakładanie elementów (Warstwy)

Głównym zadaniem Box jest wyświetlanie komponentów tak, aby mogły na siebie nachodzić. Elementy dopisane na końcu listy wewnątrz {} będą "na wierzchu".

Przykład: Zdjęcie profilowe, a na nim mała zielona kropka statusu "online".

2. Precyzyjne pozycjonowanie (Alignment)

Box pozwala na bardzo łatwe "przypięcie" elementu do dowolnego rogu lub środka kontenera za pomocą parametru **contentAlignment**.

3. Tworzenie tła lub filtrów

Często używa się Box, aby nałożyć półprzezroczysty filtr na obrazek lub stworzyć niestandardowe tło pod grupą elementów.

Przykład kodu: Ikona powiadomień z kropką

```
Box(
    modifier = Modifier.size(64.dp).background(Color.Green),
    contentAlignment = Alignment.Center // Domyślnie wszystko na środku
) {
    // 1. Warstwa dolna: Ikona
    Icon(
        Icons.Default.Notifications,
        contentDescription = null,
        modifier = Modifier.size(48.dp)
    )

    // 2. Warstwa górna: Czerwona kropka przypięta do prawego górnego rogu
    Box(
        modifier = Modifier
            .size(15.dp)
```

```

        .background(Color.Red, shape = CircleShape)
        .align(Alignment.TopEnd) // Specjalny modyfikator dostępny tylko w Box
    )
}

```

Kluczowe parametry Box

1. **contentAlignment**: Ustawiasz go w nawiasach (). Decyduje, gdzie domyślnie lądują elementy (np. **Alignment.BottomCenter**).
2. **propagateMinConstraints**: Jeśli ustawisz na **true**, Box wymusi na swoich dzieciach, aby miały co najmniej taki sam rozmiar jak on sam.
3. Modyfikator **.align()**

Wewnątrz klamer {} możesz każdemu elementowi z osobna nadać modyfikator **.align()**, aby ustawić go w konkretnym miejscu:

- **Alignment.TopStart** (Lewy górny)
- **Alignment.Center** (Środek)
- **Alignment.BottomEnd** (Prawy dolny)
- ...i wszystkie inne kombinacje (łącznie 9 pozycji).

• Pole tekstowe edycyjne (**OutlinedTextField** oraz **TextField**)

Różnica między **TextField** a **OutlinedTextField** sprowadza się niemal wyłącznie do stylistyki wizualnej i sposobu, w jaki prezentują się na ekranie. Oba komponenty działają identycznie pod względem kodu (mają te same parametry, jak **value** czy **onValueChange**).

1. Wygląd wizualny

- **TextField** (Filled): Ma wypełnione tło (zazwyczaj szare) i tylko dolną linię (border-bottom). Wygląda bardziej "płasko". Często kojarzony ze starszym stylem Material Design (Material 2).
- **OutlinedTextField**: Nie ma wypełnienia tła (jest przezroczyste), za to posiada pełną ramkę (obramowanie) dookoła. Wygląda lżej i nowocześniej, zgodnie z wytycznymi Material 3.

2. Zachowanie etykiety (Label)

- **TextField**: Gdy zaczniesz pisać, etykieta unosi się do góry, pozostając wewnątrz wypełnionego obszaru tła.
- **OutlinedTextField**: Gdy zaczniesz pisać, etykieta unosi się i "wskakuje" w linię ramki, przecinając ją. Tworzy to bardzo estetyczny efekt wizualny.

@Composable

```

fun LoggingTextField() {
    // Stan przechowujący tekst
    var textState by remember { mutableStateOf("") }

    OutlinedTextField(
        value = textState,
        singleLine = true, // To sprawia, że pole ma tylko jedną linię
        maxLines = 1,      // Dodatkowe zabezpieczenie (limit linii)
        //typ klawiatury
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
        onValueChange = { newText ->
            // 1. Aktualizujemy stan, aby tekst pojawił się w polu
            textState = newText
            // 2. Wysyłamy wiadomość do Logcat
            Log.d("Tag", "Użytkownik wpisał: $newText")
        },
        label = { Text("Wpisz coś...") },
    )
}

```

```

        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp)
    )
}

```

Przykład konwersji tekstu do wartości całkowitej:

```

@Composable
fun SimplestIntExample() {
    var textValue by remember { mutableStateOf("") }

    Column(modifier = Modifier.padding(16.dp)) {
        // 1. Pole tekstowe pobierające znaki
        OutlinedTextField(
            value = textValue,
            onValueChange = { textValue = it },
            label = { Text("Wpisz liczbę") },
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number)
        )

        // 2. Próba zamiany na Int
        // toIntOrNull() spróbuje zrobić liczbę, jeśli się nie uda (np. puste pole) -
        // poda 0
        val number = textValue.toIntOrNull() ?: 0

        Spacer(modifier = Modifier.height(16.dp))

        // 3. Wykorzystanie liczby do obliczeń
        Text(text = "Liczba jako Int: $number")
        Text(text = "Liczba pomnożona przez 2: ${number * 2}")
    }
}

```

Zadania

Zadanie 1: Personalizowany Witacz

Stwórz aplikację, która posiada pole tekstowe do wpisania imienia oraz przycisk "Przywitaj mnie".

- Po wpisaniu tekstu w OutlinedTextField, tekst powinien być na bieżąco zapamiętywany.
- Po kliknięciu przycisku Button, pod spodem powinien pojawić się tekst: "Witaj, [Imię]! Miło Cię widzieć!".
- Jeśli pole jest puste, po kliknięciu przycisku powinien pojawić się komunikat: "Proszę wpisać imię!".

Zadanie 2: Kalkulator

Napisz aplikację, która pozwoli wprowadzić dwie wartości do dwóch edycyjnych pól tekstowych umieszczonych obok siebie oraz cztery przyciski (poniżej pól tekstowych, wszystkie obok siebie), które kolejno będą dodawać, odejmować, mnożyć oraz dzielić te liczby.

Ustaw odpowiednie etykiety dla przycisków. Wynik powinien być wyświetlany poniżej przycisków w polu tekstowym.

Zadanie 3: Konwerter Jednostek (Poziom: Średni)

Zaprojektuj prosty konwerter walut (np. PLN na EUR) lub jednostek miary (np. Kilometry na Mile).

- Aplikacja powinna mieć jedno pole edycyjne na samej górze.
- Poniżej, w jednym wierszu, powinny znajdować się dwa przyciski: "Na Euro" oraz "Na Dolar".

- Kliknięcie przycisku pobiera wartość z pola tekstowego, przelicza ją (użyj stałego kursu, np. 1 EUR = 4.30 PLN) i wyświetla wynik w dużym polu tekstowym na dole.
- **Dodatkowo:** Spraw, aby przyciski w miały taką samą szerokość.

Zadanie 4: Karta Kontrolna Smart Home

Stwórz panel sterowania jasnością żarówki.

- Użyj Box, aby stworzyć duży kwadrat (np. 200.dp) reprezentujący żarówkę.
- Wewnątrz Box umieść Text, który wyświetla aktualny poziom jasności (w %).
- Pod kwadratem umieść pole tekstowe, w którym użytkownik wpisuje liczbę od 0 do 100.
- Obok (lub pod spodem) dodaj przycisk "Ustaw jasność".
- **Kluczowy element:** Po kliknięciu przycisku, kolor tła Twojego Box powinien stać się tym jaśniejszy, im wyższa jest wpisana liczba (możesz użyć `Color.Yellow.copy(alpha = ...)`).